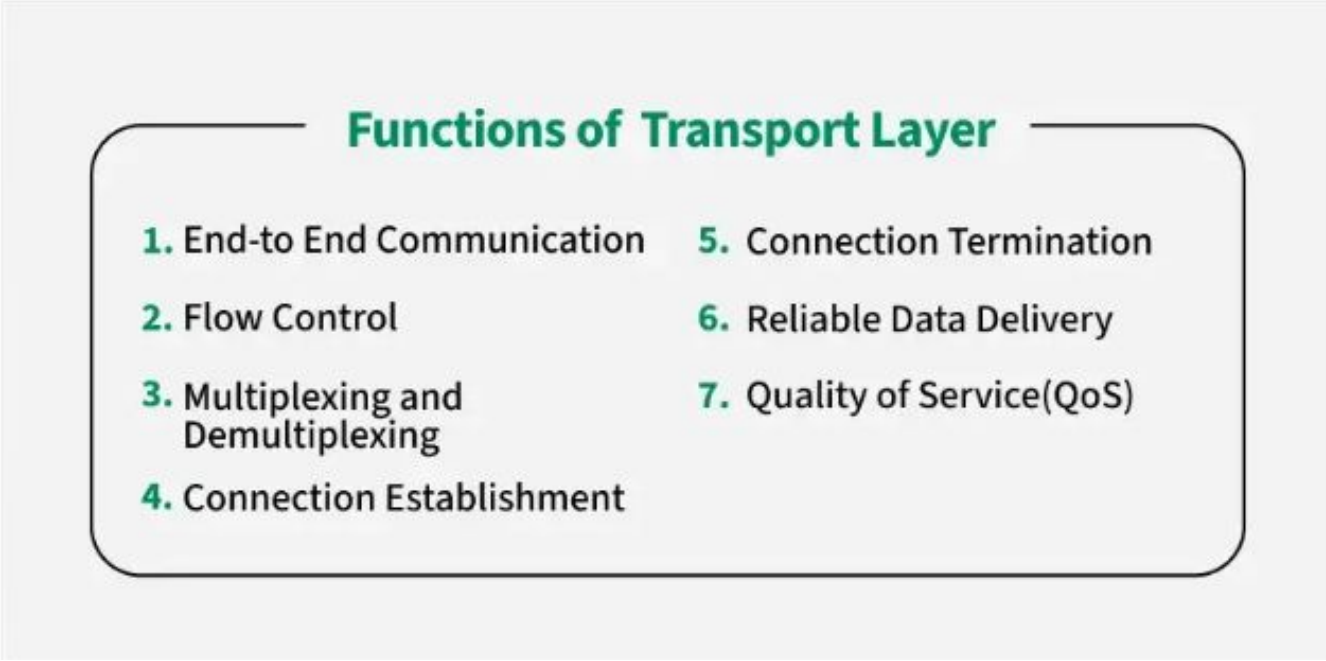
Transport Layer

The Transport Layer is the OSI Layer 4 responsible for providing reliable, efficient, and ordered end-to-end communication between applications on different hosts.

- Operates at Layer 4 (Transport Layer) of the OSI model
- Enables process-to-process communication using port numbers
- Ensures reliability through error control, sequencing, and retransmission
- Supports flow control to prevent receiver overload
- Uses protocols like **TCP, UDP, and SCTP** for different application needs



Transport Layer Protocols

Transport Layer Protocol uses different protocol for the better communication between two ends uses of protocol may differ from specifications. Below mention are some protocols used in Transport Layer

1. Transmission Control Protocol(TCP)

- **TCP** is connection-oriented Protocol.
- TCP is reliable protocol.
- As TCP is connection-oriented protocol, so first the connection is established between two ends and then data is transferred and then the connection is terminated after all data being sent.

2. User Datagram Protocol (UDP)

- UDP is not reliable protocol
- The protocol UDP is connectionless.
- When speed and size are more important than security and dependability, this kind of protocol is employed.
- The data from the higher layer is supplemented with transport-level addresses, checksum error control, and length information by UDP, an end-to-end transport level protocol.
- A user datagram is the packet that the UDP protocol generates.

Transport Layer Protocols

Let's start discussing the layer 4 protocols with the Transmission Control Protocol (TCP). The following diagram shows the TCP header format. Pay special attention to the source and destination port fields, which are 16 bits long. This gives 2^{16} (65,536) possible port numbers.

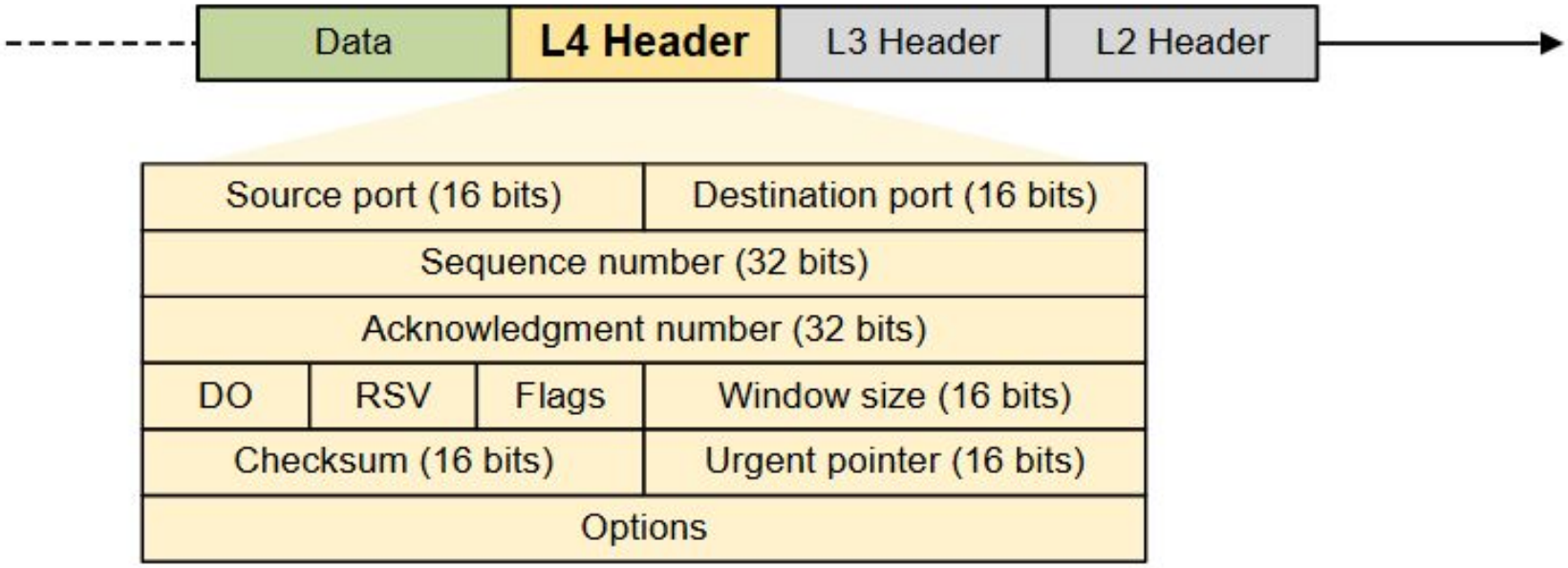
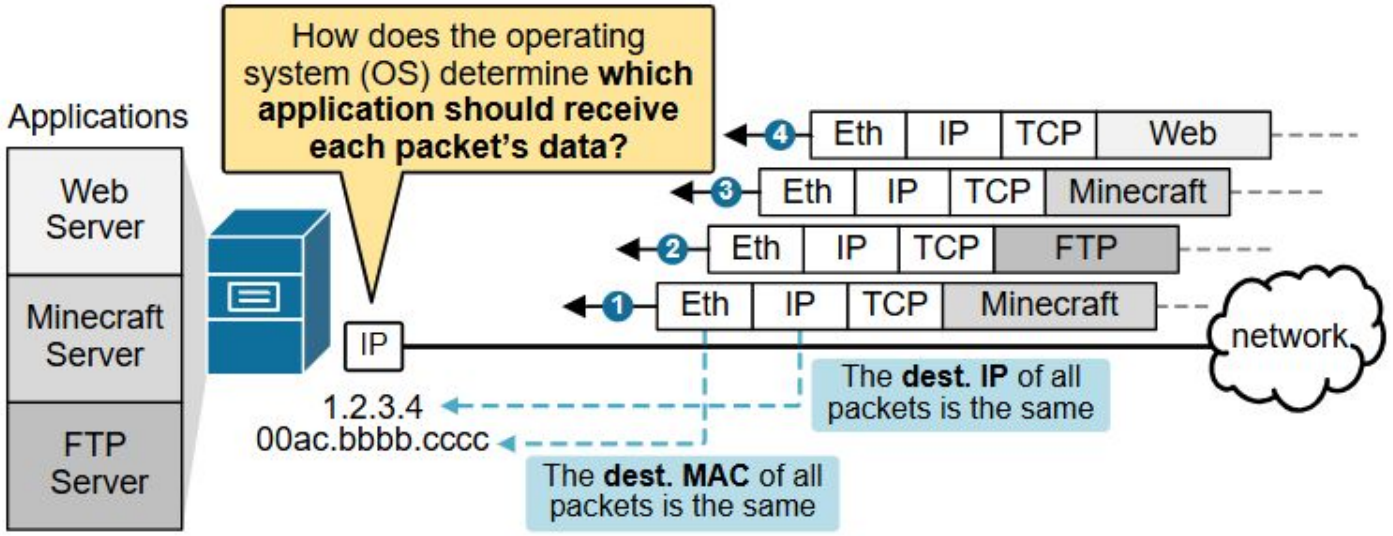


Figure 1. TCP Header Format.

Transport Layer Protocols

Why do we need TCP port numbers?

To understand the layer 4 protocols, let's first discuss why we need to have TCP/UDP ports in the layer 4 header. Let's look at the example shown in the diagram below. We have a server that runs many applications at the same time. For example, it runs a Web server, a Minecraft server, and an FTP server **all at once**.

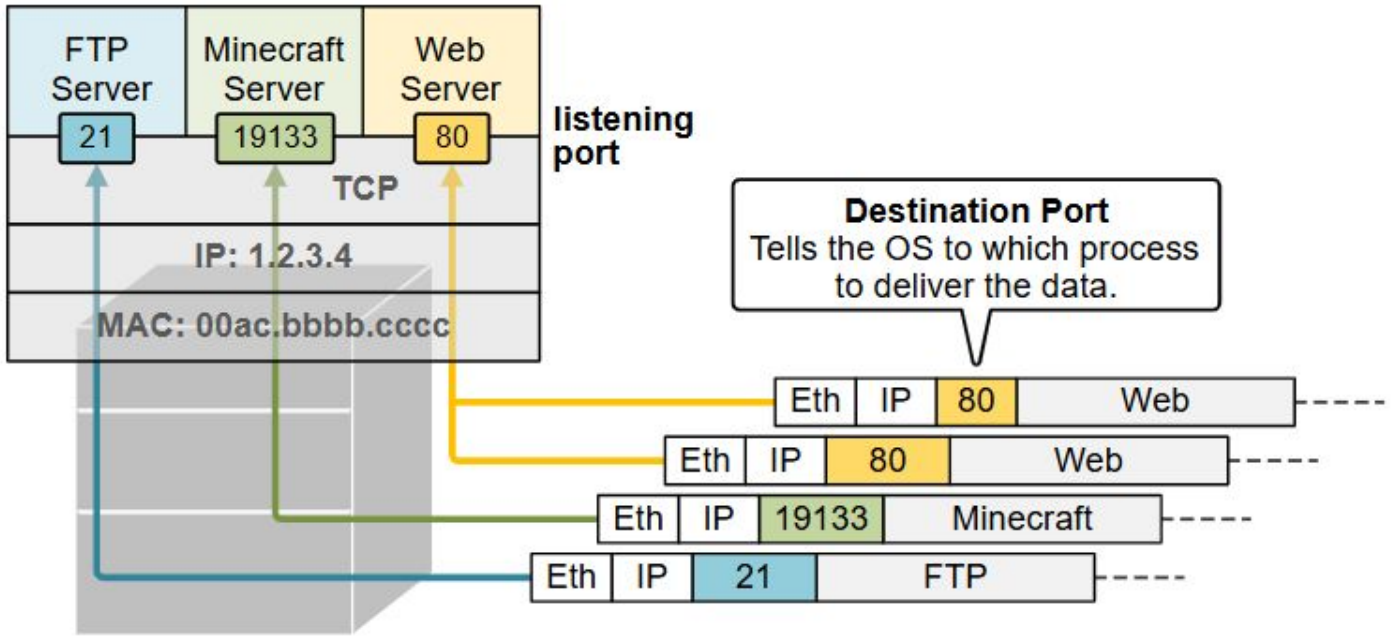


Now, let's see what happens when the server receives many packets at once. All packets have the same destination MAC address (the MAC of the server's NIC card) and the same IP address (the IP address of the server's NIC card). Hence, based on the Layer 2 and Layer 3 headers, the server's operating system **cannot understand which application the packet's data belongs to**.

Transport Layer Protocols

Multiplexing using port numbers:

Multiplexing using port numbers allows devices to handle many connections at the same time. When packets arrive at a device, the port number helps the device determine which application should receive the data, as shown in the diagram below.



Multiplexing using port

Transport Layer Protocols

Each process that runs on the server "listens" for incoming data on a specific TCP port, as shown in the output below. For example, the Web Server listens on port 80, while the FTP Server listens on port 21.

```
Microsoft Windows [Version 11.0.25621.5339]
(c) Microsoft Corporation. All rights reserved.

C:\> netstat -a -n

Active Connections
  Proto  Local Address          Foreign Address        State
  TCP    1.2.3.4:21             0.0.0.0:0              LISTENING
  TCP    1.2.3.4:80             0.0.0.0:0              LISTENING
  TCP    1.2.3.4:443            0.0.0.0:0              LISTENING
  TCP    1.2.3.4:19133          0.0.0.0:0              LISTENING
```

When packets arrive at the server, the operating system examines the destination port number in the Layer 4 header. It forwards the data **to the process that listens to that port number**. For instance, the OS delivers the packets with destination port 80 to the web server process.

Transport Layer Protocols

Well-known, Registered, and Dynamic Ports

The next important thing is that the port numbers in the layer 4 header of packets are not random. The Internet Assigned Numbers Authority (IANA) manages the global assignment of port numbers. It divides them into three main groups:

- **Well-Known Ports (0–1023):** These are assigned by IANA for common services like HTTP (port 80) or FTP (port 21). Getting a new port in this range requires a strict review.

- **Registered (User) Ports (1024–49151):** These are also assigned by IANA, but the process is easier than for well-known ports.

- **Dynamic or Ephemeral Ports (49152–65535):** These are not assigned by IANA. They are used by client apps for temporary connections. The OS picks these ports when needed.

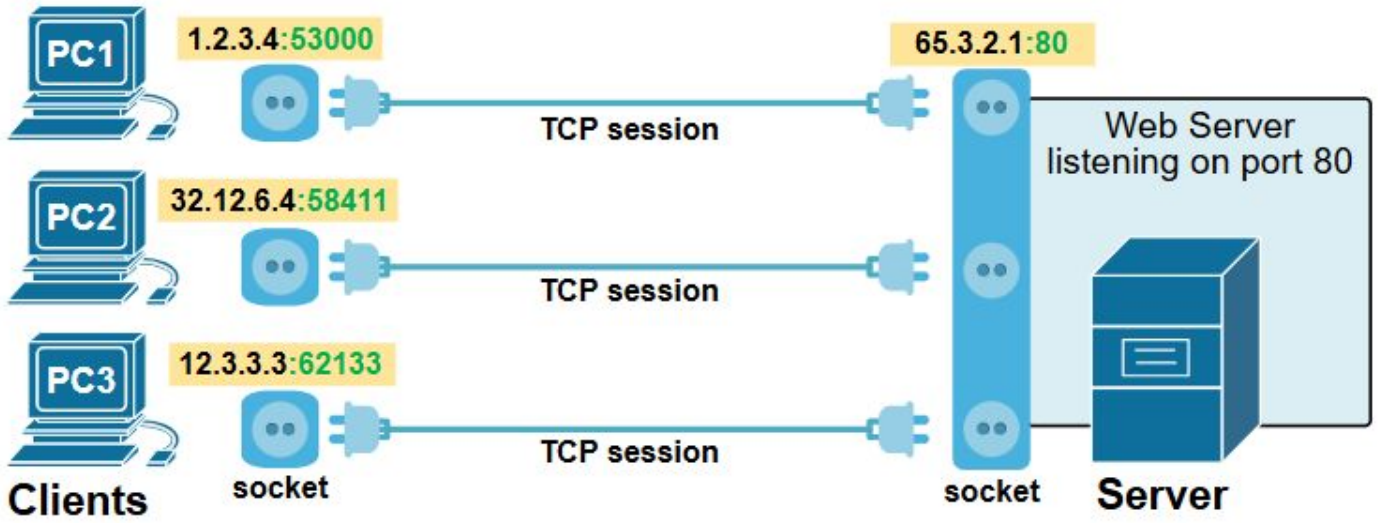
For example, your computer might open several applications at once. Each app will use a different dynamic port so the OS can keep the connections separate. Servers, on the other hand, use well-known or user ports so clients know where to connect.

Transport Layer Protocols

What is a socket? What is a TCP Session?

Now that we have seen what the port number is, let's take a closer look at another term: a socket. A socket is the combination of an IP address and a port number. It **identifies one end of a connection** between two devices. For example, 1.2.3.4:53000 is a socket where 1.2.3.4 is the IP address and 53000 is the TCP port.

Sockets allow devices to know where to send and receive the actual data in the packet's payload. A pair of sockets defines a unique TCP connection called **a TCP session**, as shown in the diagram below.



Transport Layer Protocols

For example, the TCP session between PC1 and the server can be uniquely identified as highlighted below:

```
Microsoft Windows [Version 11.0.25621.5339]
(c) Microsoft Corporation. All rights reserved.

C:\> netstat -a -n

Active Connections
  Proto  Local Address          Foreign Address        State
  TCP    1.2.3.4:53000          65.3.2.1:80        ESTABLISHED
```

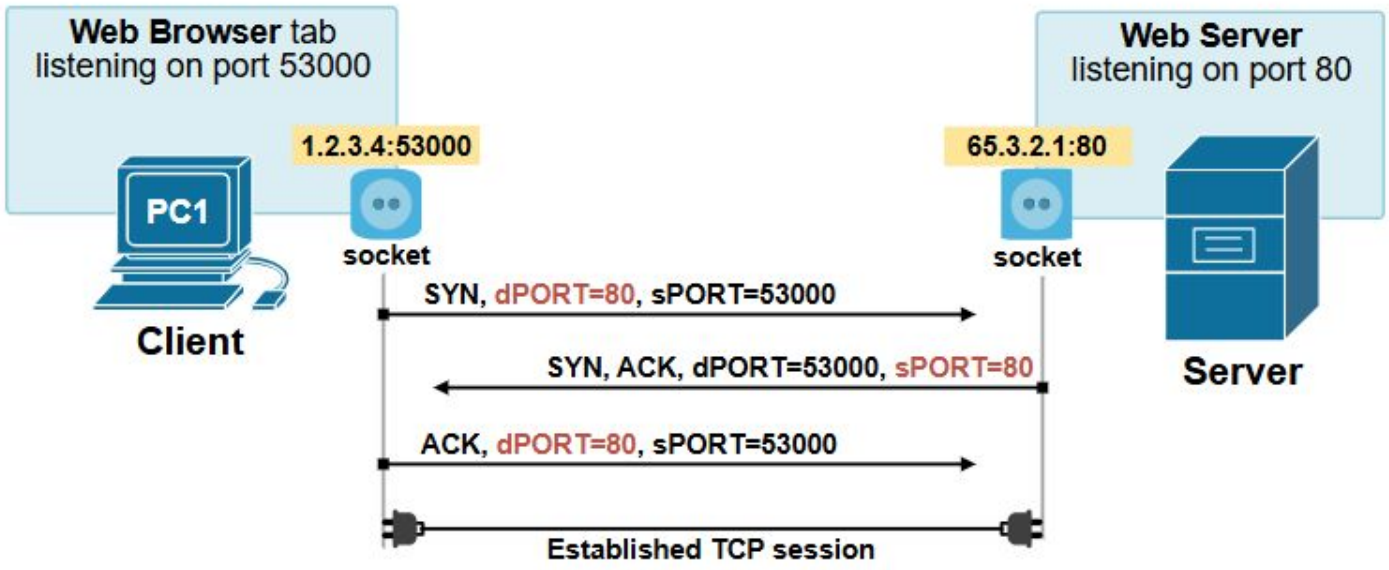
- **The client** is the application that sends a request. For example, a web browser requests a web page. Notice that the client uses a randomly assigned **dynamic port number**.

- **The server** is the application that waits for requests and provides a service. For example, a web server that sends back the requested web page. Notice that the server uses a **well-known port number**.

Transport Layer Protocols

Establishing a TCP session:

Now let's shift our focus to the connection establishment process. During this step, the client and the server agree on port numbers and set up the sequence and acknowledgment numbers so they can track the data they send and receive.
The setup process is called **the three-way handshake**. The following diagram visualizes how it works:



Three-way handshake

Transport Layer Protocols

Establishing a TCP session:

- The client sends a SYN message to the server to start the connection.
- The server replies with a SYN-ACK message to agree and acknowledge.
- The client answers back with an ACK message.

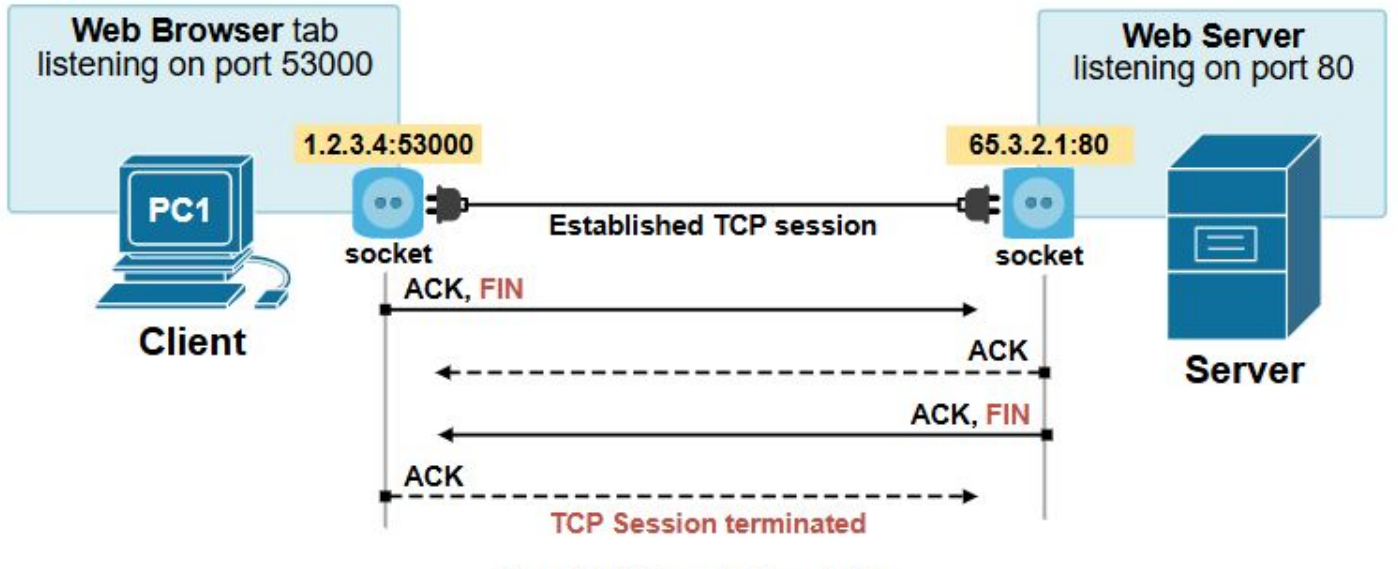
Once this handshake is done, data can start flowing between the client and server. The connection is defined by the sockets on both sides.

Transport Layer Protocols

TCP Session Termination:

When it's time to end the connection, TCP uses a four-step process called termination, as shown in the diagram below:

- One device sends a FIN message to indicate that it has finished sending data.
- The other device replies with an ACK.
- That device then sends its own FIN message to close the connection.
- The first device replies with a final ACK.



Transport Layer Protocols

3. Stream Control Transmission Protocol (SCTP)

- Many Internet applications use SCTP to perform transport layer duties, similar to User Datagram Protocol (UDP) and Transmission Control Protocol (TCP).
- On top of a connectionless packet network like IP, SCTP is a dependable transport protocol that facilitates data transfer over the network in scenarios involving one or more IP addresses.

TCP	UDP
Connection-Oriented	Connectionless
Guaranteed Transmission	No Guarantee
Critical applications	Real-Time applications
Slower	Faster
20 bytes header	**8 bytes header**
Three-Way Handshake	No Handshake Mechanism
Acknowledgment Mechanism	No Acknowledgment
HTTP, HTTPs, FTP, SMTP, etc.	DNS, DHCP, TFTP, SNMP, etc.